

Most Likely Interview Questions Preparation Session of ECP & DS

Date: 28/09/2021

**** Key Points:**

- Basics
- focus on quality & not on quantity
- Body Language - dressing sense, presentation skills, eye contact, learning attitude etc....

eDESD - Embedded C Programming

Q. Explain compilation and execution flow of C program?

Ans:

- by default compiler writes an addr of main() function as an entry point function into the elf header inside _start() routine.
- linux commands to check contents of an executable file:
\$readelf -h program.out
\$objdump

Q. What is the difference between declaration & definition? Explain in context of functions & variables.

Ans:

- In C, if we want to use any variable / function in a out program, we have to first defined it before its use

functions:

- function declaration / function prototype must contains:
return type function_name(signature of the function);

return type : any primitive / non-primitive type

function_name : identifier

signature of the function : list of formal parameters / arguments of the function

- no. of parameters
- data types of parameters
- order of parameters
- names of parameters (optional)

e.g.

int addition(int, int);

float addition(int, float, int);

function definition:

has two things

1. function header : first line of function definition is referred as function header, and it is same as function prototype, except two things:

- there is no semi-colon at the end of the line
- name of parameters is mandatory

2. function body : implementation of actual function logic inside curly braces { } .

function calling:

- we cannot define one function inside another function
- we can call any function from inside any other function, function can be called from inside the same function as well ==> recursion
- to give call to any function, min we required 2 things:
 1. name of the function : an addr of first instruction of the function
 2. function call operator : ()

```
void print_message(void);
```

function call:

```
int main(void){  
    print_message( );  
    -----  
}
```

stack cleanup ==> to destroy function activation record of called function, this is done either by **calling function** or **called function** ==>

function calling conventions:

c calling convention

__cdecl

__stdcall

__pascal

etc....

- whether arguments to be pushed onto stack from left to right or from right to left

variable declaration & definition:

int num;//globally defined = declaration as well as definition

int num = 99;//declaration, definition as well as initialization

extern int i;//core declaration ==> when compiler reads this line it does not allocate any memory for variable i, it assumes that variable i of type of int is defined somewhere else globally either in the same file or in another file in the same project.

Q. What is a significance of storage classes? Which Storage classes are there in C? Explain static in context of variables and functions.

- there are total 4 storage classes

1. auto i.e. automatic storage class
2. register storage class
3. extern storage class
4. static storage class

- storage classes in C decides 4 things about variables & functions:

1. by default initial value : either 0 or GV
2. location : either main memory or CPU registers
3. scope of the variable/function : local / global / file
4. lifetime : throughout the function/block/program

- static variables can be initialized only once, whereas can be modified multiple times.

- static functions defined inside one source file cannot be accessed i.e. called from inside any function which is inside another file.

Q. What do you mean by dangling pointer and memory leakage? How to detect and avoid it?

Q. What is the difference between passing argument by value and by address? Which type of arguments cannot be passed by value?

Q. Which are function calling conventions in C? What is its significance?

Q. What do you mean by structure member alignment, padding and data packing?

Q. What is an Array? What are limitations of it? Why array index begins with zero?

Q. Write a function to simulate string reverse and check whether string is palindrome or not?

```
char *string_copy( const char * src, char *dest){  
  
}
```

```
char *string_reverse(char *str)  
{  
    char *left = str;  
    char *right = str;  
    char *temp = str;  
  
    //move right pointer towards right
```

```

while( *right )
    right++;

right--;

while( left <= right ){
    char t = *left;
    *left = *right;
    *right = t;

    left++;
    right--;
}

return str;
}

```

Q. What is the difference between macro and function?

Q. What is the difference between structure and union?

Q. What is pointer ? Why pointers?

Q. What is function pointer? With example, explain how to declare and define function pointer?

- **function pointer** is a pointer variable in which we can store an addr of function (i.e. function name itself an addr of the function), so that we can pass one function as an argument to any other function.

return type (* fptr_pointer_name)(argument_list);

int addition(int, int);

int (* fptr)(int, int) = addition;

OR

int (* fptr)(int, int);
fptr = addition;

example:

float addition(int, float, int);

float (*fptr_addition) (int, float, int) = addition;

Q. What do you mean by variadic functions?(i.e. variable arguments list functions).

`printf( )`
`scanf( )`

Q. What are different Bitwise operators, and explain how they operates with example?

- there are 6 bitwise operators in C:

1. bitwise AND (`&`)
2. bitwise OR (`|`)
3. X-OR (`^`)
4. negation (`~`)
5. left shift operator (`<<`)
6. right shift operator (`>>`)

- bitwise operators can be applied only on int and char type of variables.
- it operates on bits of value of variable.

Data Structures:

Q 1. What is time complexity and space complexity? What is best case, worst case and average case time complexity?

Ans:

Time complexity of an algo is the amount of time i.e. computer time it needs to run to completion.

Space complexity of an algo is the amount of space i.e. computer memory it needs to run to completion.

- **best case time complexity** : if an algo takes min amount of time to run to completion.

- **worst case time complexity** : if an algo takes max amount of time to run to completion.

- **average case time complexity** : if an algo takes neither min nor max amount of time to run to completion.

Example: linear search

Q 2. Implement quick sort & merge sort. (paper work)

Q 3. Write a program to convert infix expression into postfix and then solve it?

Q 4. How to display a singly linear linked list in reverse order (without modifying contents)?

Q 5. How to reverse a singly linear linked list using recursion and without recursion?

Q 7. How to check if given singly linear linked list is palindrome or not? Time complexity should be $O(n)$ and space complexity should be $O(1)$.

Ans:

step-1: traverse the linked list till middle node (we can find middle node by using two pointers `slow_pointer` & `fast_pointer`, after every iteration `slow_pointer` moves towards next node, whereas `fast_pointer` moves towards its next to next node, and when `fast_pointer` reached at last node/second last node, `slow_pointer` will reached till middle node).

step-2: reverse the second half linked list

step-3: compare data part of each node of half list from first node till middle node with each node in second half reversed list, if data part of all the nodes are matches then we can say linked list is palindrome.

Q 8. Implement add last, traverse and search function in a generic (for any data type) doubly linked list.

- by using void pointer -> generic pointer

Q 9. Implement binary search tree in-order and post-order using recursion and without recursion?

Q 10. What are different methods of implementing graph? Explain with diagram and code snippet.

SunBeam